INFOTACT SOLUTIONS

OmniSight: 14-Day Engineering Log

Autonomous Multimodal UI Self-Healing & RPA Agent · Daily Sprint Report

Internship Role: Associate L1. Intern (Generative AI) | Candidate: Prajwal Sam Jaggili

Project Domain:	Multimodal AI & RPA	**Target Tech Stack:**	FastAPI, React, Playwright, Gemini VLM
Reporting Period:	14 Consecutive Sprint Days	**GitHub Integration:**	PyGithub Automated PR Bot
Project Status:	Fully Integrated & Production-Ready	**Supervising Firm:**	Infotact Solutions (Bengaluru)

EXECUTIVE PROJECT SUMMARY

14-DAY CORE MILESTONE ARCHITECTURE

Phase / Timeline	Primary Objective & Core Systems Built	Status
Days 1 - 3 Architecture & Headless Automation	Repository scaffolding, Playwright browser engine, mock test app	[·] COMPLETED
Days 4 - 5 API Gateway & Multimodal Vision	FastAPI asynchronous server, SQLite persistence, Gemini VLM integration	[·] COMPLETED
Days 6 - 7 Action Engine & Mid-Project Audit	CSS code synthesis, markdown diff extraction, milestone audit passed	[·] VERIFIED
Days 8 - 9 ReAct Loop & GitHub PR Automation	Agentic DOM self-healing, PyGithub autonomous branch & PR publication	[·] COMPLETED
Days 10 - 11 React Dashboard & Visual Diff Studio	Vite dashboard, live scan polling, side-by-side screenshot diffs	[·] COMPLETED
Days 12 - 13 Design Studio & Settings Control	Interactive CSS sandbox, token editor, multi-model selector panel	[·] COMPLETED
Day 14 End-to-End Orchestration & Delivery	Full-stack integration, live system validation, production delivery	[·] DELIVERED

Day 1 — Repository Setup, Architecture Blueprint & Environment Config

Phase 1: Project Scaffolding & Architecture Design

- Studied Infotact Solutions project specifications and defined the multi-tier agent architecture (Playwright + Gemini VLM + LangChain ReAct + PyGithub).
- Created project repository structure: backend/ (FastAPI), frontend/ (React + Vite), and mock-app/ (target testing suite).
- Configured environment templates (.env.example), dependency manifests (requirements.txt, package.json), and Docker container configuration.
- Established SQLite database schema foundations with SQLAlchemy engine for tracking scans, visual bug anomalies, and PR logs.

Verified Deliverable: Initial project scaffolding, Git repository initialized, architecture documentation written.

Day 2 — Mock Target Application & Injected Visual Anomaly Suite

Phase 1: Mock Application & Testbed Ground Truth

- Built an interactive mock e-commerce shopping web application (mock-app/) with product cards, cart drawer, checkout modal, and responsive navigation.
- Deliberately injected realistic visual bugs across breakpoints: overlapping typography, bad color contrast (WCAG violations), and button clipping outside viewport.
- Constructed ground_truth.json defining precise coordinates, element selectors, and severity classifications for automated testing benchmarks.
- Validated local asset serving and confirmed baseline rendering across desktop, tablet, and mobile device screen viewports.

Verified Deliverable: Functional mock e-commerce application with 8 calibrated visual layout defects and benchmark ground truth file.

Day 3 — Headless Browser Automation & Viewport Screenshot Pipeline

Phase 1: Browser Automation & Playwright Engine

- Implemented BrowserService using Playwright (backend/services/browser/) to execute headless Chrome navigation.
- Configured multi-viewport automated capture: Desktop (1920x1080), Tablet (768x1024), and Mobile (375x812).
- Engineered DOM state extractors to capture raw HTML trees, computed CSS layouts, and element bounding boxes alongside screenshots.
- Added robust network idle waiting, screenshot compression (JPEG/WebP), and local artifact storage management.

Verified Deliverable: Automated headless capture engine outputting synchronized DOM trees and multi-viewport screenshots.

Day 4 — FastAPI Scan Endpoints, SQLite Storage & ORM Models

Phase 1: Backend API Gateway & Database Layer

- Built FastAPI asynchronous server routing in backend/main.py with endpoints for triggering scans (/api/scans), fetching run status, and querying results.
- Created SQLAlchemy models for ScanRun, VisualBug, RemediationAction, and PullRequestHistory with SQLite thread pooling.
- Implemented auto-cleanup migrations on startup to handle orphaned or pending scans from previous worker executions.
- Tested CORS configuration, request validation schemas (Pydantic), and database query indexing.

Verified Deliverable: Working backend REST API with persistent SQLite database storing scan history and bug metadata.

Day 5 — Gemini Multimodal Vision Integration & Prompt Engineering

Phase 2: Vision-Language Model Integration

- Developed VLMService (backend/services/vlm/) integrating Google Gemini multimodal vision APIs.
- Engineered structured spatial reasoning prompts instructing the VLM to inspect screenshots alongside DOM node hierarchies.
- Implemented JSON-enforced schema outputs requiring exact bug locations (bounding box coordinates), issue types, severity levels, and root causes.
- Created fallback handling and rate-limiting retry strategies with exponential backoff for API resilience.

Verified Deliverable: VLM analysis service producing structured JSON visual bug reports from raw UI screenshots.

Day 6 — Action Engine, Regex Isolators & CSS/React Code Synthesis

Phase 2: Action Engine & Code Patch Extraction

- Constructed the Action Engine backend module to parse natural language VLM remediation suggestions into pristine executable code blocks.
- Implemented strict Markdown code block extractors (```css, ```jsx, ```html) and CSS syntax sanity validators.
- Built an AST and rule-based filter to prevent destructive overwrites and verify responsive CSS rules (flexbox, grid, z-index, clamp font sizes).
- Mapped identified UI bugs directly to actionable remediation diff objects with before/after style representations.

Verified Deliverable: Code extraction pipeline converting VLM visual critique into isolated, syntactically verified CSS/HTML patches.

Day 7 — Mid-Project Review Audit: Autonomous Vision Detection Benchmark

Phase 2: Mid-Project Audit & Pipeline Validation

- Conducted the required Mid-Project Milestone Audit: passed automated test suites against mock-app injected visual defects.
- Verified that the VLM accurately flagged 100% of target anomalies (hidden checkout button, text overlap, low contrast banner) without manual intervention.
- Validated that isolated patch objects cleanly targeted the corrupted CSS classes without corrupting neighboring layout elements.
- Refactored concurrency models to allow non-blocking asynchronous scan processing using FastAPI BackgroundTasks.

Verified Deliverable: Mid-project milestone certification: successful end-to-end screenshot-to-patch detection audit.

Day 8 — LangChain ReAct Self-Healing Loop & In-Memory DOM Mutation

Phase 3: Agentic ReAct Orchestrator

- Implemented the AgentOrchestrator (backend/services/agent/) utilizing a ReAct (Reasoning + Acting) loop pattern.
- Created an in-memory DOM patch applicator using Playwright to inject generated CSS fixes directly into the live page session.
- Built the verification loop: after patch injection, capture an immediate post-remediation screenshot and trigger VLM visual re-inspection.
- Added self-correction logic: if the bug persists after re-inspection, the agent reflects on the failure and generates an alternative CSS fix.

Verified Deliverable: Autonomous ReAct self-healing feedback loop with iterative patch verification and regression prevention.

Day 9 — PyGithub Integration, Automated Branching & PR Generator

Phase 3: GitHub Autonomous Pull Request Integration

- Developed GitHubService (backend/services/github/) with authenticated PyGithub integration for target repositories.
- Engineered automated branch creation workflows (e.g., fix/omnisight-visual-patches-run-id) from the repository default branch.
- Generated rich, markdown-formatted Pull Request descriptions containing visual bug summaries, severity badges, and code diff previews.
- Attached visual evidence: uploaded before/after screenshots directly into the PR description to give human reviewers instant verification.

Verified Deliverable: Autonomous GitHub PR creator capable of branching, committing patches, and submitting complete PRs with visual diffs.

Day 10 — React Dashboard Setup, Vite Server & Live Polling Engine

Phase 4: Frontend Dashboard Architecture & Real-Time Monitoring

- Scaffolded modern React/Vite dashboard (frontend/) configured with Lucide icons, glassmorphic styling, and dark theme design system.
- Created responsive layout components: Sidebar navigation, TopBar status pills, and Live Scan Overview cards.
- Implemented custom polling hooks (useScanStream) to synchronize with FastAPI scan execution states in real-time.
- Built metric summary counters: Scanned Pages, Identified Defects, Auto-Generated Fixes, and Pull Requests Published.

Verified Deliverable: Interactive React dashboard frontend connected via API proxy to the live FastAPI backend server.

Day 11 — Before/After Visual Comparison Studio & Code Diff Explorer

Phase 4: Visual PR Reviewer & Interactive Diff Viewer

- Engineered interactive Before vs. After split-screen and slider comparison views for inspected viewports.
- Built an embedded code diff component with syntax highlighting for reviewing proposed CSS/JSX modifications.
- Added single-click human-in-the-loop controls: 'Approve & Create GitHub PR' and 'Dismiss Finding'.
- Implemented bounding box highlight overlays pinpointing exact bug locations over captured screenshot canvases.

Verified Deliverable: Visual PR Reviewer interface providing QA engineers with side-by-side screenshot comparisons and diff approvals.

Day 12 — Interactive Design Studio & Live CSS Injection Sandbox

Phase 4: Design Studio & Interactive Visual Editor

- Developed the dedicated DesignStudio.jsx component for real-time visual inspection and CSS playground experimentation.
- Integrated multi-viewport preview switcher (Desktop 1920px, Tablet 768px, Mobile 375px) within the browser interface.
- Created live style token adjustment panels (spacing, typography, color palettes, border radius) with instantaneous preview updates.
- Implemented export functions for saving approved design system adjustments directly into project asset repositories.

Verified Deliverable: Design Studio UI allowing live interactive design modifications, viewport testing, and design token tuning.

Day 13 — Configuration View, VLM Model Switcher & Performance Tuning

Phase 4: Settings Management, API Keys & Multi-Device Optimization

- Built SettingsView.jsx supporting dynamic configuration of Gemini API keys, GitHub Access Tokens, and Target Repository endpoints.
- Implemented dynamic VLM model selector (Gemini 2.5 Flash, Gemini Pro, GPT-4o Vision) with latency/cost indicator badges.
- Optimized backend image handling with thumbnail caching and asynchronous streaming to eliminate UI lag during multi-screenshot scans.
- Fixed database locking issues in SQLite by enforcing timeout configurations and isolated session lifecycles.

Verified Deliverable: Complete settings administration panel, credential management, and responsive frontend optimization.

Day 14 — Full-Stack Integration, Live Server Orchestration & Demonstration Audit

Phase 4: Full System Integration, End-to-End Verification & Production Readiness

- Conducted comprehensive end-to-end regression tests: launched backend (uvicorn on port 8000), frontend (Vite on port 5173), and mock app.
- Executed full autonomous run: trigger scan -> multi-viewport screenshot capture -> Gemini VLM visual bug detection -> CSS patch extraction -> GitHub PR generation -> real-time display in dashboard.
- Verified system stability, zero console errors, clean shutdown hooks, and seamless proxy routing between Vite and FastAPI.
- Prepared final demonstration assets, source code archives, and verified readiness for the final portfolio review.

Verified Deliverable: Fully integrated, production-ready OmniSight autonomous self-healing RPA agent and executive dashboard.